

Lecture – 11

UML Basics: Inheritance, Association, Aggregation and Composition

Afsara Tasneem Misha

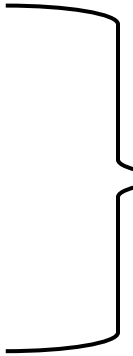
Lecturer

Department of CSE

Daffodil International University

Contents

- **UML Basics**
  - Generalization (Inheritance) – **IS-A Relationship**
  - Association
  - Aggregation
  - Composition

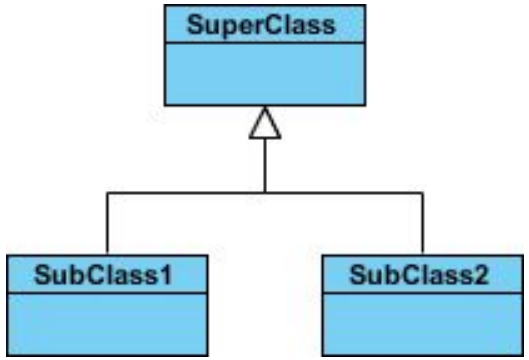


– **HAS-A Relationship**

Topic - 1 : Inheritance (Generalization)

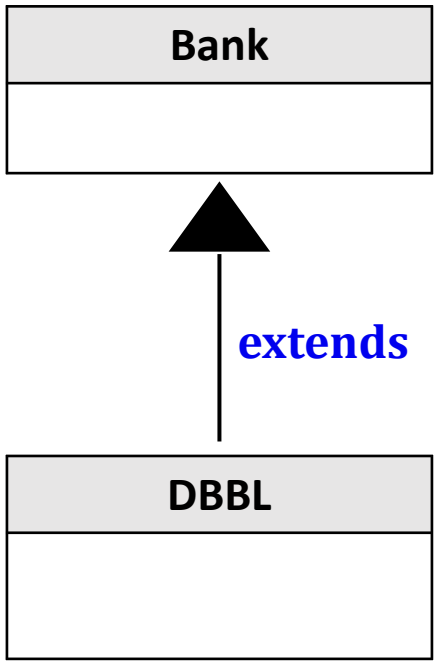
- A generalization is a taxonomic relationship between a **more general** classifier and a **more specific** classifier.

- Represents an "**is-a**" relationship.



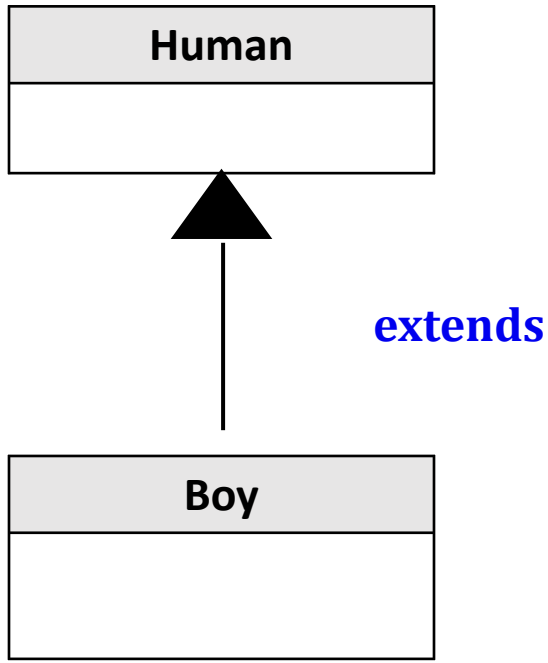
- **SubClass1** and **SubClass2** are **specializations** of **SuperClass**.

IS-A relationship



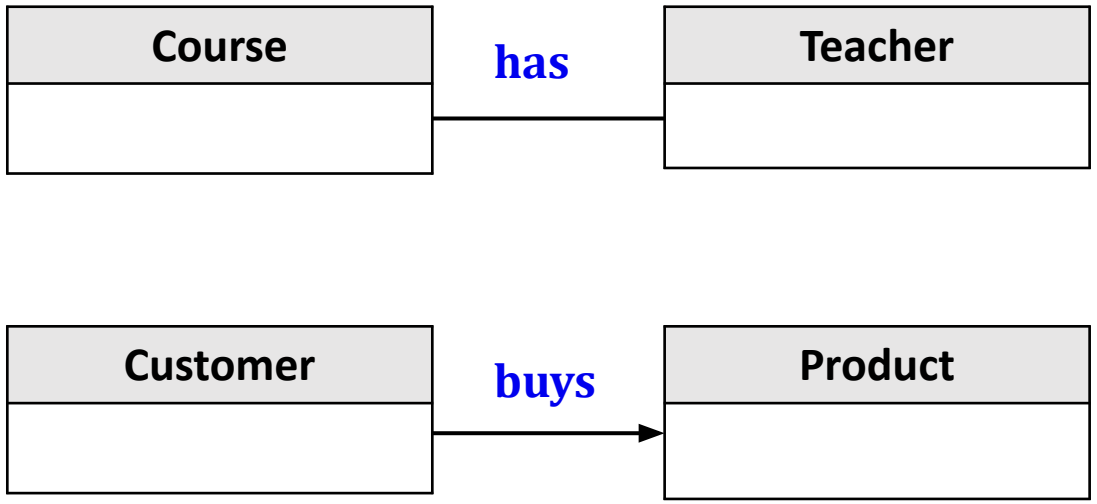
Super // Generalization

Sub // Specialization



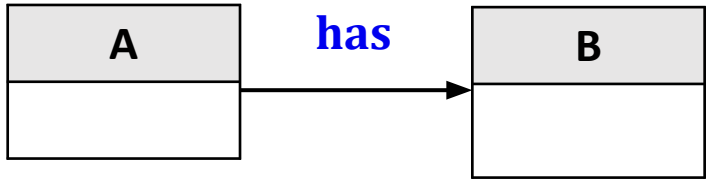
Topic - 2 : Association

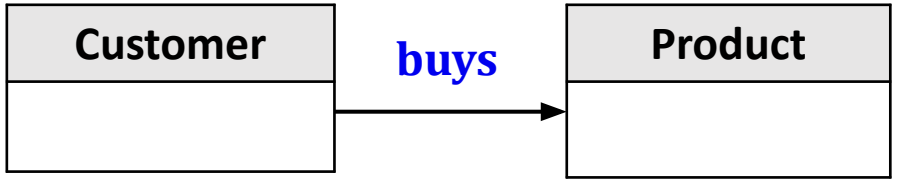
- **Relationship** between **two separate classes** which establishes through their Objects. .
- Also known as **“has-a"** relationship.
- Each class is **independent**. They can exist without each other.



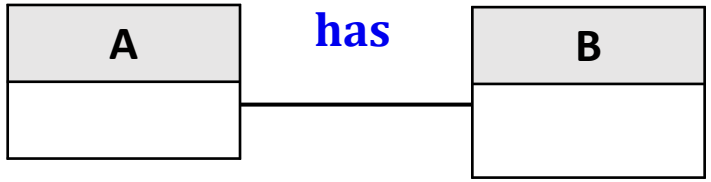
Two types of Association

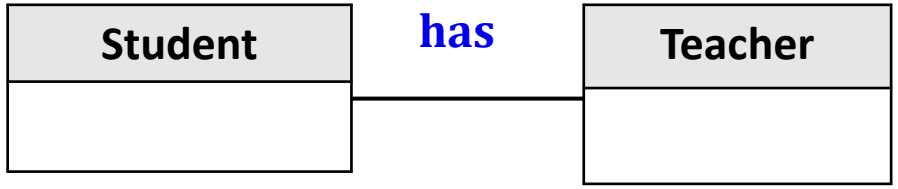
- **Unary Association:** Class A has Class B; But Class B does not have Class A





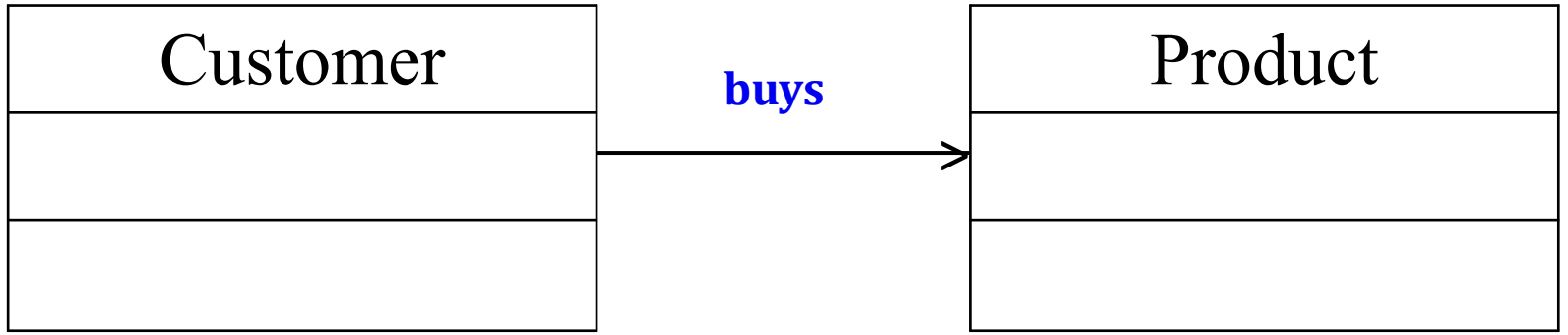
- **Binary Association:** Both Classes know about each other.





Two types of Association

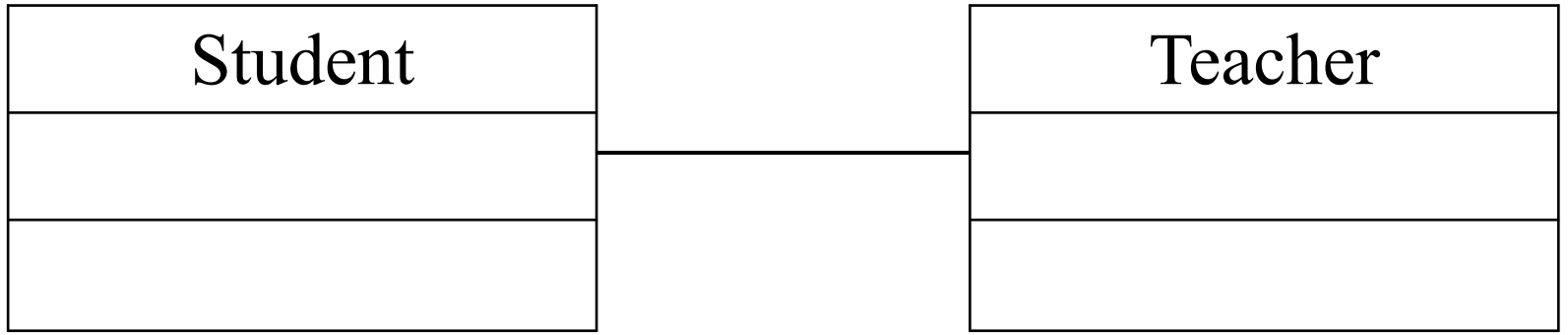
1. Unary Association



Customer Class will have the object of Product

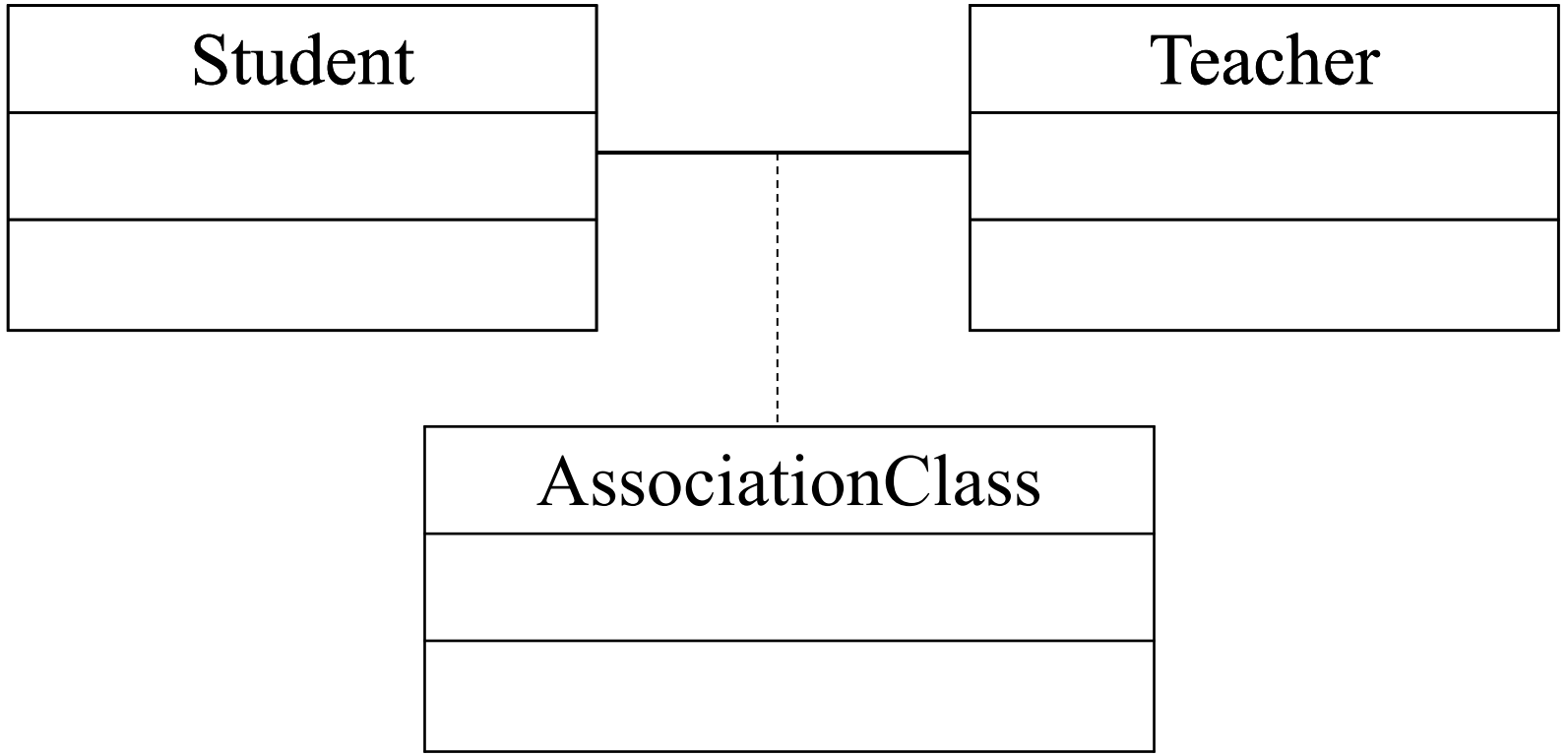
Two types of Association

2. Binary Association



Two types of Association

2. Binary Association (Solved by Association Class)

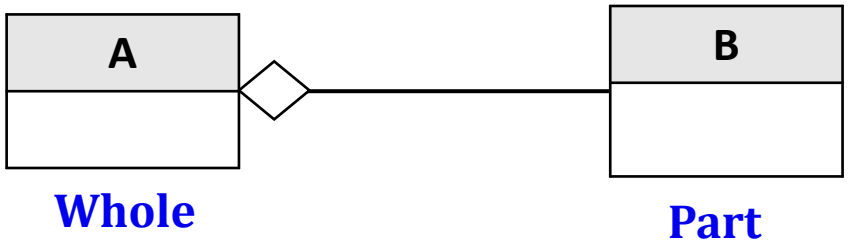


Topic - 3 : Aggregation and Composition

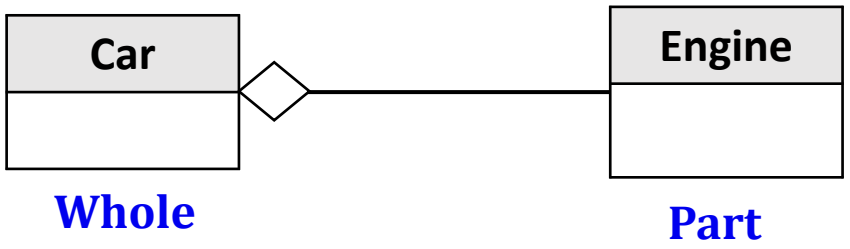
[Whole/Part Relationship]

3.1: Aggregation

- **Specified** form of **Association**
- **Whole/Part** Relationship

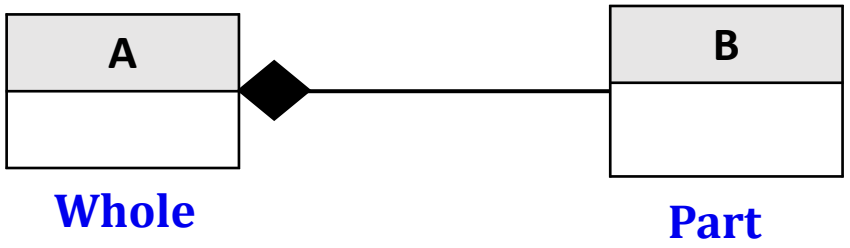


- Class B **is a part of** Class A.
- Both Classes are **independent**. Part class **can exist** without Whole Class.

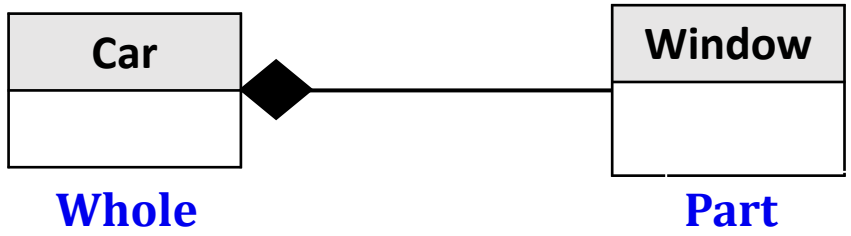


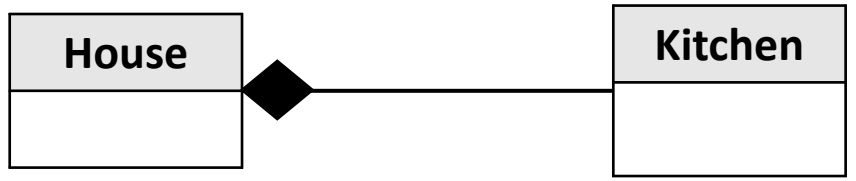
3.2: Composition

- **Specified (Stronger)** form of **Association**
- **Whole/Part** Relationship

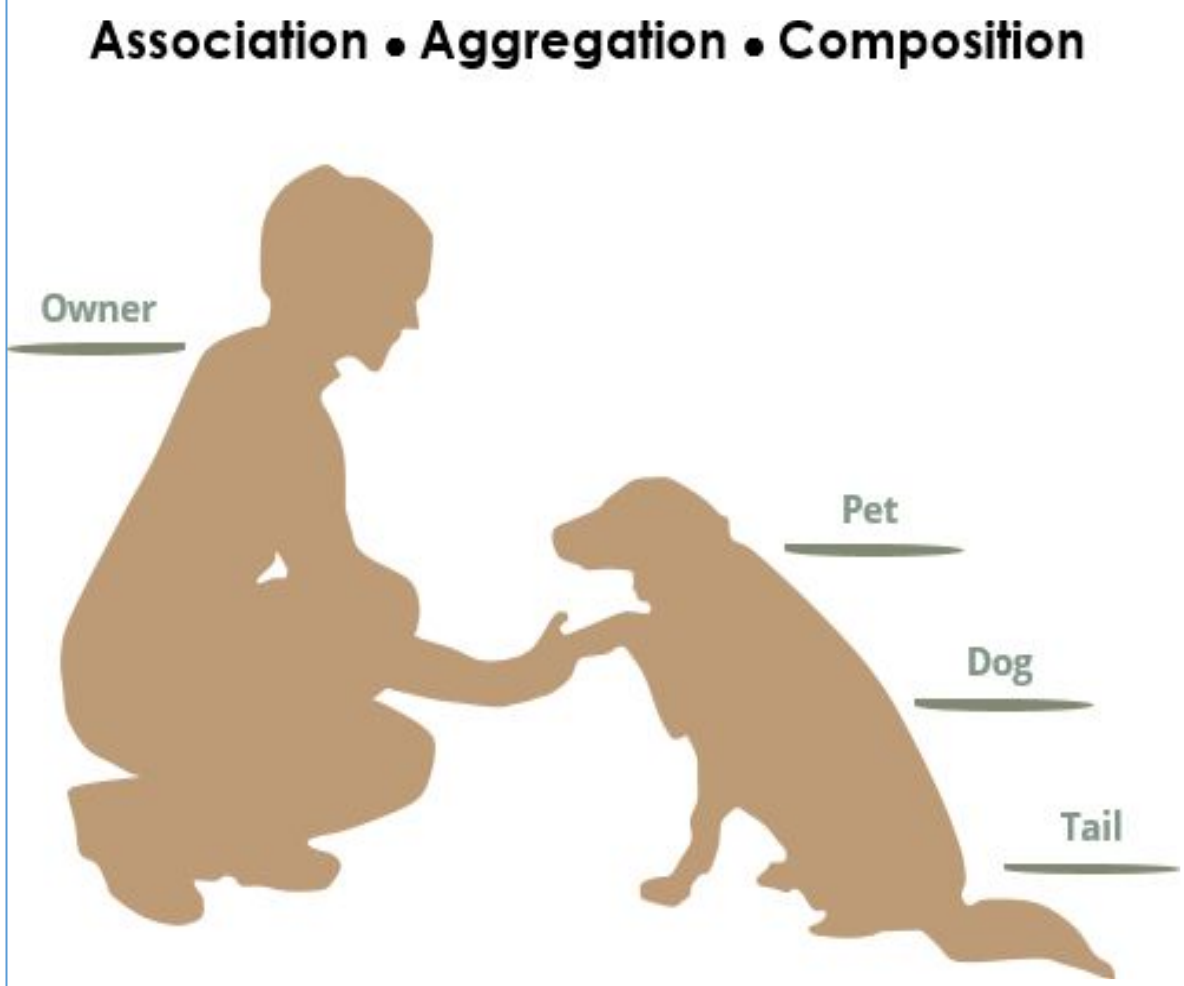


- Class B **is a part of** Class A.
- The **existence** of Part class **depends on** Whole Class.





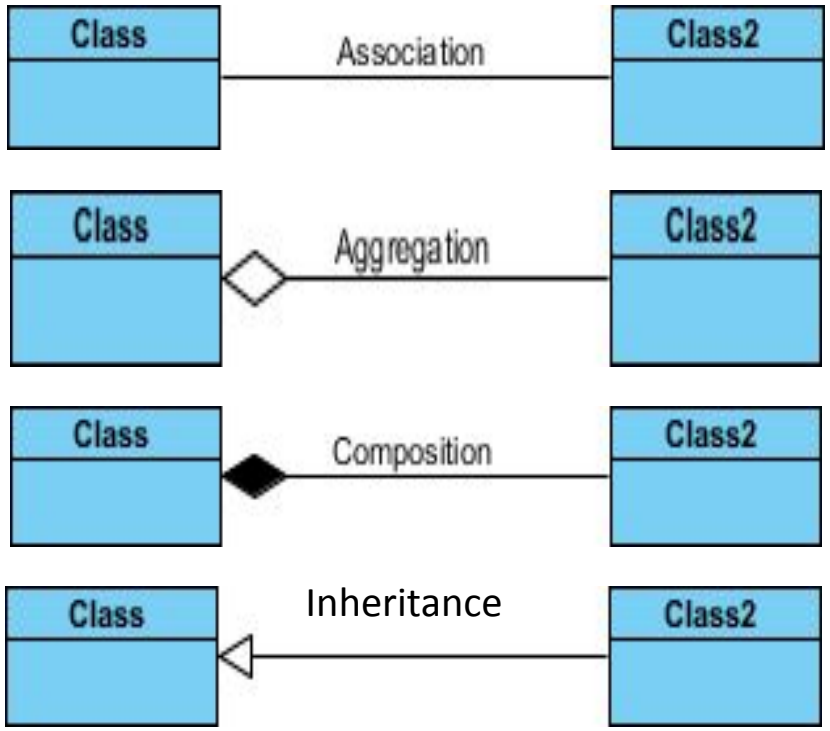
The figure below shows the three types of association connectors: association, aggregation and composition

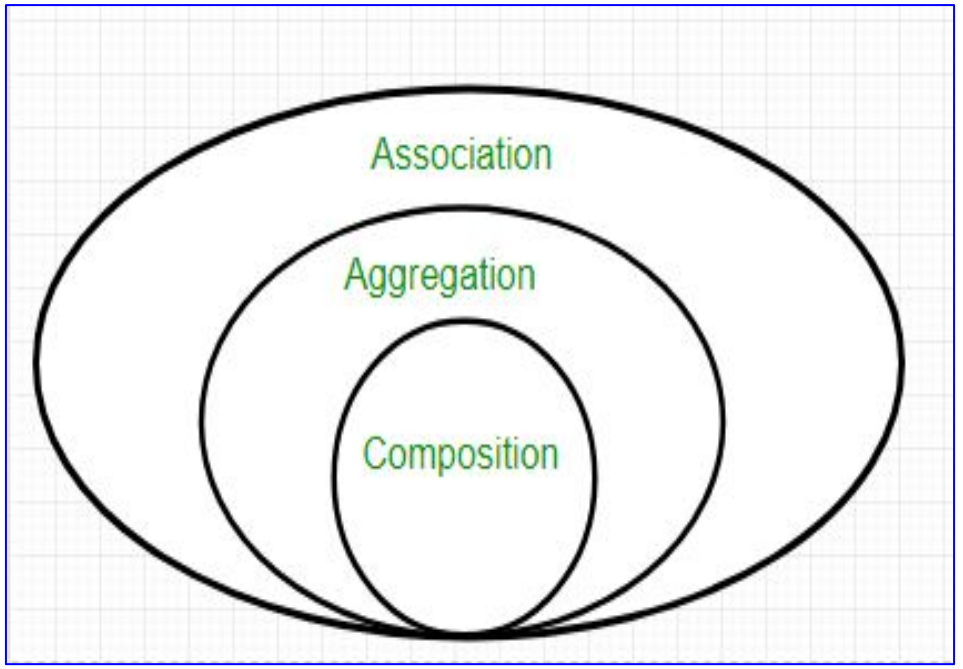


We see the following relationships:

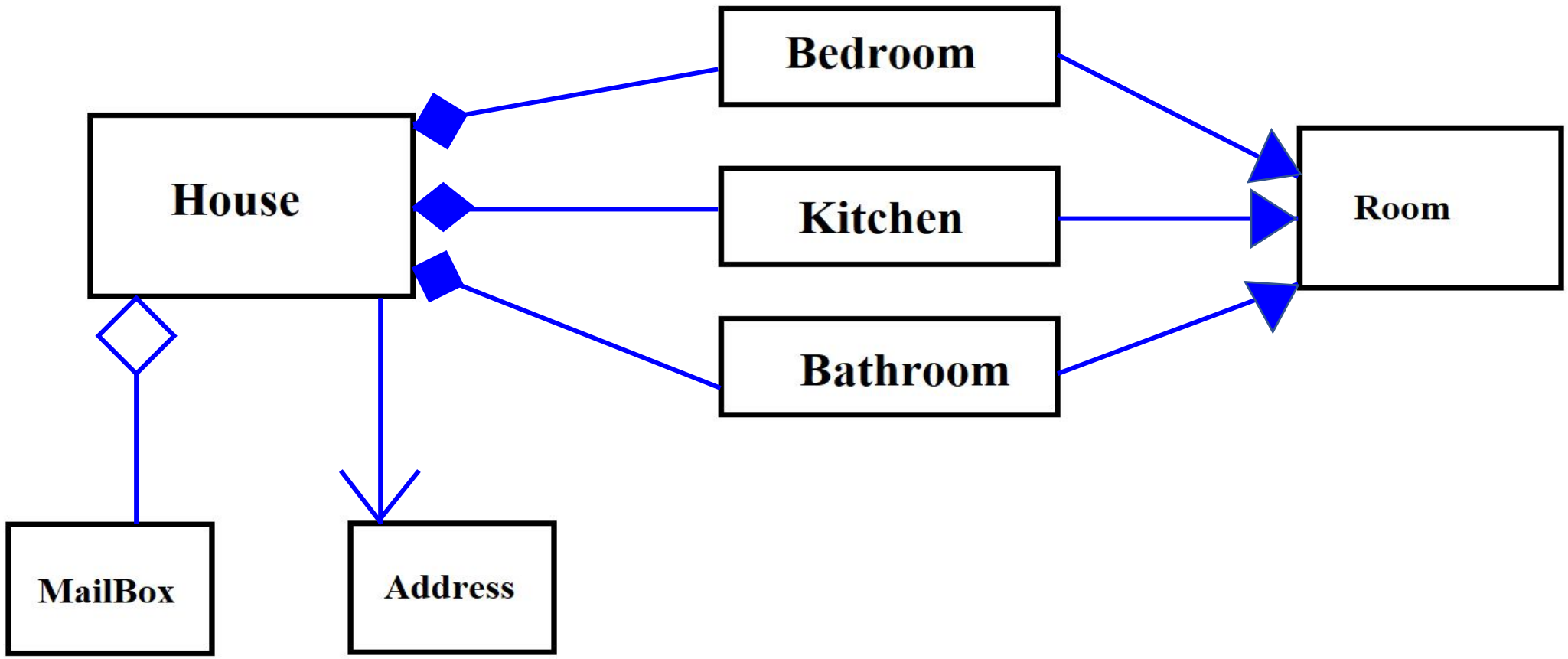
- **owners** feed **pets**, **pets** please **owners** (**Association**)
- a **tail** is a part of both **dogs** (**Aggregation / Composition**)
- a **dog** is a kind of **pet** (**Inheritance / Generalization**)

UML Notation





Example - 1



Example – 2: Description to UML

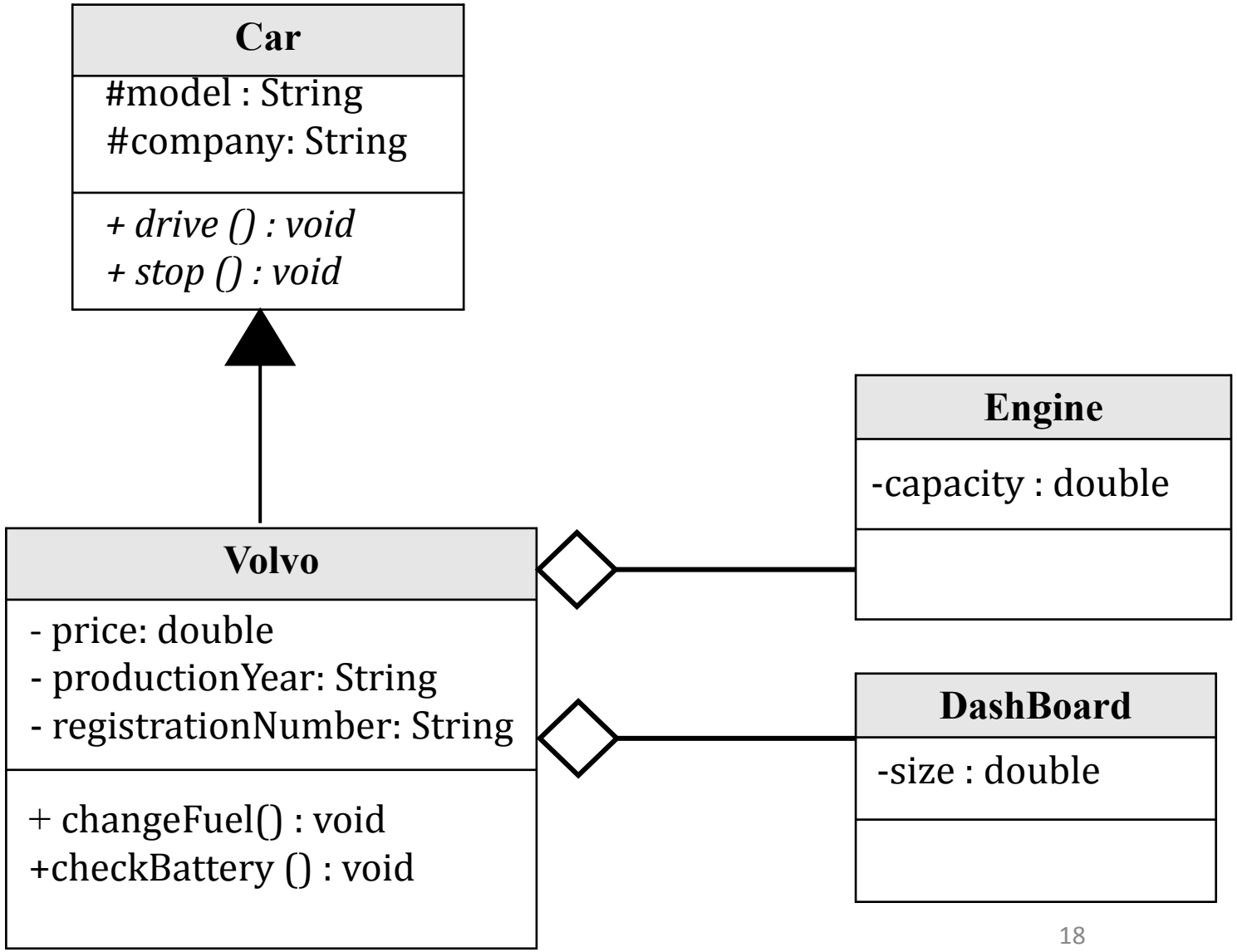
- Volvo is a Car.
- Every car has model and company of type string.
- Volvo has price of type double, productionYear and registrationNumber of type string.
- Volvo also has Engine and DashBoard.
- Engine has capacity of type double.
- DashBoard has size of type double.
- Car provides drive and stop as abstract service or method of type void.
- Volvo also provides changeFuel and checkBattery service or method of type void.

Example – 2: Description to UML

- Volvo **is a** Car.
- Every car **has** **model** and **company** of type **string**.
- Volvo **has** **price** of type **double**, **productionYear** and **registrationNumber** of type **string**.
- Volvo also **has** **Engine** and **DashBoard**.
- Engine **has** **capacity** of type **double**.
- DashBoard **has** **size** of type **double**.
- Car **provides** **drive** and **stop** as abstract **service** or **method** of type void.
- Volvo also **provides** **changeFuel** and **checkBattery** **service** or **method** of type void.

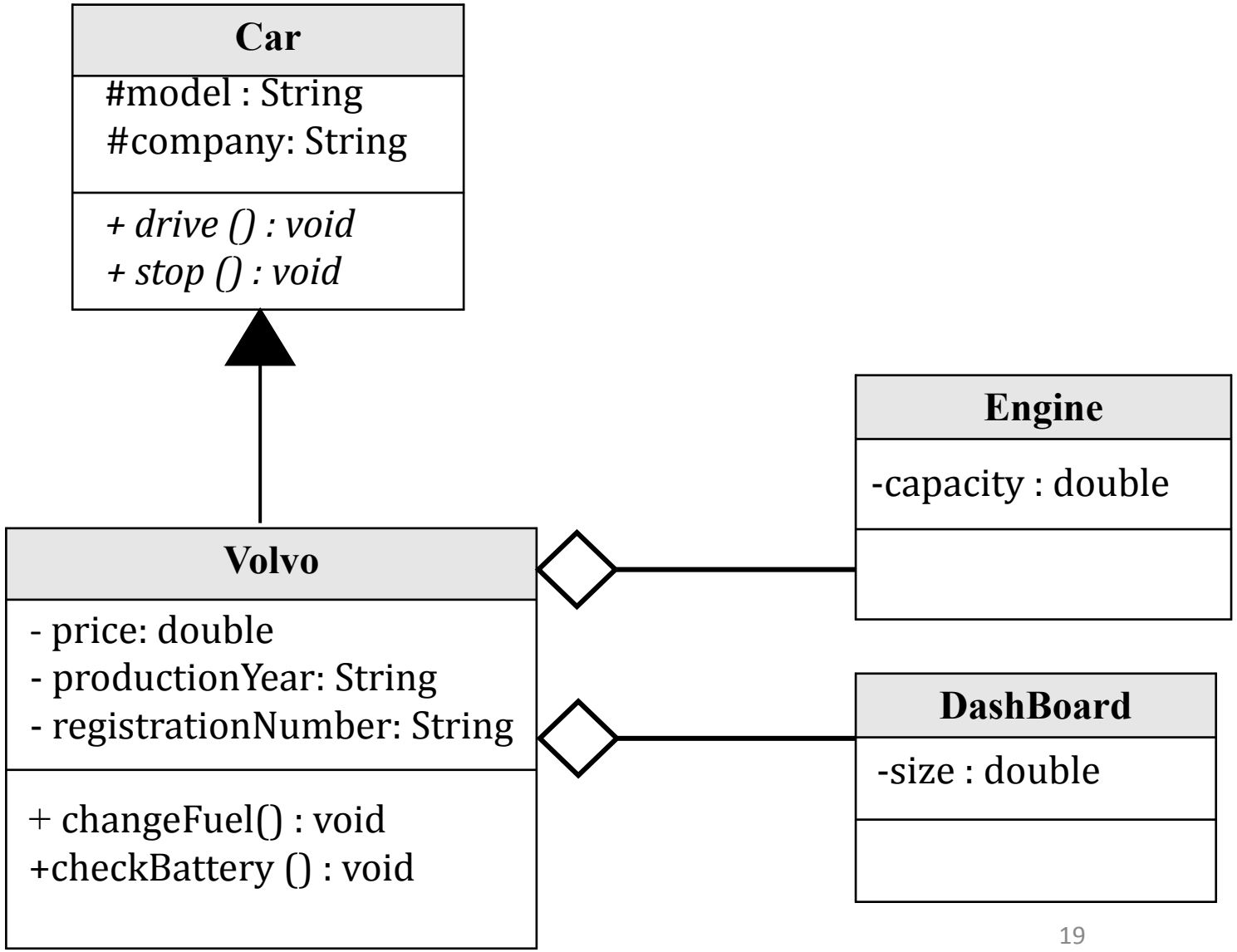
Solution of Example – 2: Description to UML

- Volvo **is a** Car.
- Every car **has** **model** and **company** of type **string**.
- Volvo **has** **price** of type **double**, **productionYear** and **registrationNumber** of type **string**.
- Volvo also **has** **Engine** and **DashBoard**.
- Engine **has** **capacity** of type **double**.
- DashBoard **has** **size** of type **double**.
- Car **provides** **drive** and **stop** as abstract **service** or **method** of type void.
- Volvo also **provides** **changeFuel** and **checkBattery service** or **method** of type void.



Solution of Example – 2: Description to UML

- Volvo **is a** Car.
- Every car **has** **model** and **company** of type **string**.
- Volvo **has** **price** of type **double**, **productionYear** and **registrationNumber** of type **string**.
- Volvo also **has** **Engine** and **DashBoard**.
- Engine **has** **capacity** of type **double**.
- DashBoard **has** **size** of type **double**.
- Car **provides** **drive** and **stop** as abstract **service** or **method** of type void.
- Volvo also **provides** **changeFuel** and **checkBattery service** or **method** of type void.



Example – 3: Try Yourself

- Apple is a fruit
- Apple has color and origin
- Apple has Sticker
- Sticker has logo

Thank you!

